Generation, retrieval-augmented systems, and where this leaves you

COCO and the Part V corpora

LECTURE 24 | GÉRON CH. 15–16

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Where we are

Lecture 23 built a catalogue you can search with a sentence. Two encoders trained apart put text and images in spaces with nothing between them, and ranked no better than chance; one dual encoder, trained contrastively on pairs, on the unit sphere and with a temperature, put the right entry first for 75.0% of 200 queries.

- Some entries have **no usable description** — nothing for a text index to encode
- Some queries are **ambiguous**, and a ranking is a poor answer to a question the user has not finished asking

> **BOTH ARE REPAIRED BY GENERATION**
>
> A multimodal model can *write* the missing description. And a generator with the retriever attached can answer the ambiguous query in words, grounded in what the catalogue actually contains.

Today

1. Captioning: filling the holes in the catalogue
2. Retrieval-augmented generation — Lecture 20's retriever joined to Lecture 18's generator, and the failures that belong to the join rather than to either half
3. **The course, as one argument** — and what it did not cover
4. **Design exercise** — a developer-tools company's in-product documentation assistant, in nine steps, with the pitfalls and how to address them

Open Lecture 24 in Colab

The hole in the index

60 of 200 catalogue entries have no description. A text index cannot rank a document that does not exist, so those entries score exactly 0% — structurally, not statistically.

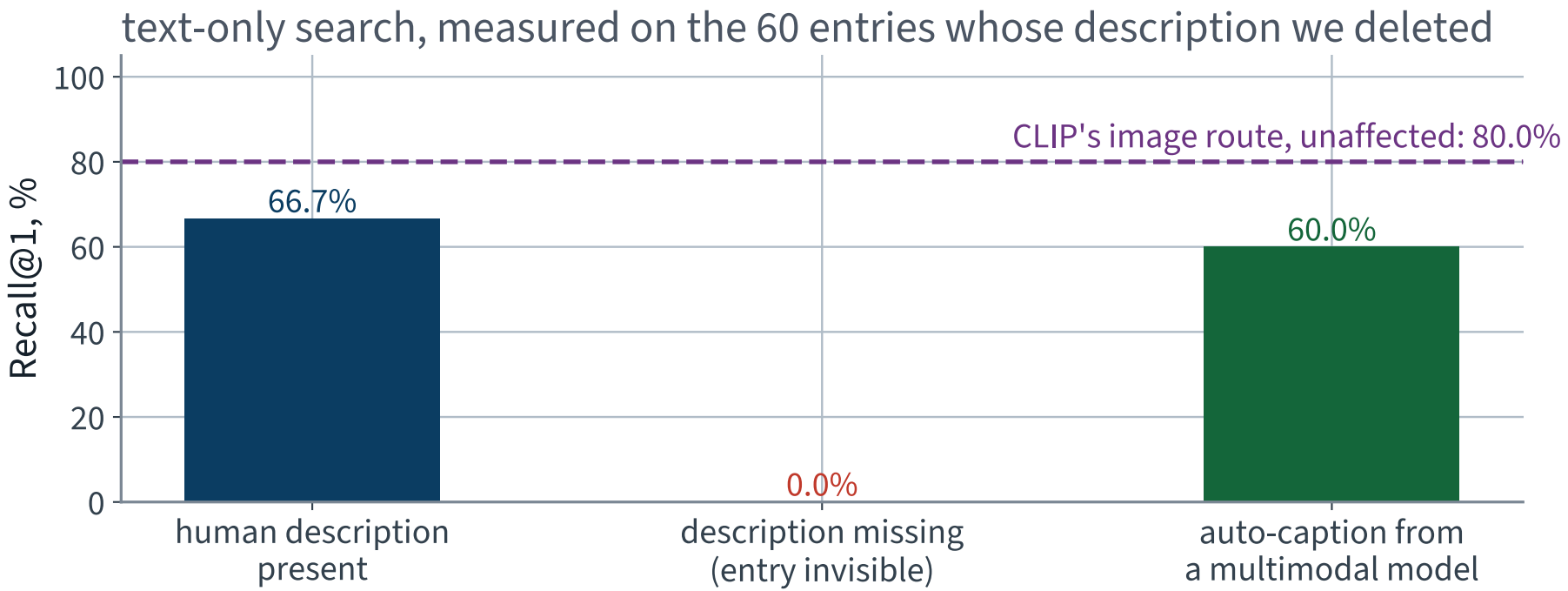
> **THE IDEA**
>
> We have a model that maps images to text: Lecture 23 measured it running in the image-to-text direction. Use a model built for *generating* the text rather than retrieving it, and write the missing descriptions.

The captioner

Component	What it is
Image encoder	a vision transformer — Chapter 16
Text decoder	a transformer decoder with cross-attention to the image — Chapter 15
Training	image–caption pairs, next-token prediction
Size	224 million parameters

Architecturally this is the encoder–decoder of Chapter 15 with an image encoder in place of the text one. Nothing new; a different pair of modalities in the same box.

Better than nothing, measured



From 0% to 60.00% on the 60 affected entries — against 66.67% if a person had written them.

The verdict

Both repairs lean on Lecture 23's joint space, which nothing today touches. For the first time in this course, a first guess at its number was probably *too low* rather than too high — because the failure was not in the evaluation. It was in the geometry, and the repair was somebody else's pretraining run.

> **STILL WORTH CHECKING**
>
> 75.0% over 200 candidates is not 75.0% over fifty thousand. Recall rises as the haystack shrinks, and a catalogue is a haystack that grows.

The join, drawn

1. The user asks a question in words
2. The **retriever** of Lecture 20 embeds it and returns the k most similar catalogue entries
3. Those entries are placed in the **generator**'s context, with the question
4. The generator answers *from what it was given*, and cites which entry it used

> **WHY THIS IS NOT JUST A BIGGER MODEL**
>
> The knowledge lives in the index, not in the weights. Add a product and it is answerable immediately; remove one and it stops being cited. No retraining, and the system can say *where* an answer came from — which is usually the requirement that decides whether it can be deployed at all.

What each half contributes

	Good at	Bad at
Retriever	finding the right entries, fast, over millions	saying anything; it only ranks
Generator	fluent answers, synthesis across entries	knowing anything reliably; it will produce a plausible answer regardless

Each covers the other's weakness, which is why the combination is worth more than either. It is also why the failures are interesting: they belong to **neither half**.

Failures of the join

THREE, AND NONE IS VISIBLE FROM EITHER COMPONENT

Retrieved but ignored. The right entry is in the context and the generator answers from its own weights instead.

Not retrieved, answered anyway. The retriever misses, and the generator does not say so — it produces a fluent answer with nothing behind it.

Retrieved and misread. The entry is present and correct, and the answer misstates it.

✗ Evaluating the retriever alone (recall@k) and the generator alone (fluency) can both look excellent while the system does all three. The join needs its own measurement.

Chunking, and why it decides everything

Before anything is retrieved, the corpus must be cut into units. That choice is upstream of the retriever and of the generator, and it is usually made without measurement.

- **Too small** — a chunk lacks the context that makes it answerable, and the generator gets a fragment
- **Too large** — the embedding averages several topics and matches none of them well
- **Split badly** — a sentence cut in half retrieves for neither of its halves

✗ It is a hyperparameter. Treat it as one: vary it, hold everything else fixed, and measure recall@k on the same judgements — Lecture 19's protocol, unchanged.

How many to retrieve

k trades two failures against each other.

Small k	Large k
the right entry may not be there at all	it is there, buried among distractors
the generator cannot recover	the generator may attend to the wrong one

WHY RE-RANKING EARNS ITS COST HERE

Retrieve a generous k with the cheap bi-encoder, then re-rank with the cross-encoder of Lecture 20 and pass only the top few on. High recall from the first stage, high precision from the second — which is the two-stage argument of Part V, arriving where it matters most.

How to measure the join

- **Groundedness** — is every claim in the answer supported by a retrieved entry? Check mechanically where you can
- **Attribution** — does the cited entry actually say it? A citation that does not support its claim is worse than none
- **Abstention** — when nothing relevant is retrieved, does it decline? Measure this deliberately, with queries whose answer is genuinely absent
- **End to end** — and only then, is the answer right?

> **THE CLOSED-BOOK CONTROL**
>
> Run the same questions with the retrieval turned off. Whatever the system still answers correctly, it knew already — and the difference is what retrieval actually bought. Same shape as every ablation in this course.

What the customer actually types

- “something to sit on”
- “gear for bad weather”
- “something to put flowers in”

A ranked list of pictures is a poor answer to any of these. The customer wants a shortlist *with reasons* — and a ranking has no place to put a reason.

Six such queries, written by hand, and the counts below are the notebook's run over them. There is no dataset of ambiguous queries; inventing them and saying so is the honest option.

Two options that do not work

OPTION A

Show the top ten pictures and let the customer decide. That is what we already have, and it is why they asked in words.

OPTION B

Ask a language model. It has never seen our catalogue, so anything it recommends is invented — and it will invent it fluently.

Retrieval-augmented generation is the composition of the two: **retrieve first, then generate strictly from what was retrieved.**

First, decide how you will check it

“The answer is good” is not measurable in a lecture. So give every entry a stock number and require the assistant to cite them:

> **THE METRIC**
>
> A cited SKU either exists in the catalogue or it does not. Count the citations, count the ones that exist, report the fraction. No judgement call anywhere in the loop.

This measures *grounding*, not helpfulness. Say which one you measured; they are not the same thing and only one of them is cheap.

Retrieve, then generate

```
q = unit(clip_text(["something to sit on"]))      # (1, 512)
top5 = np.argsort(-(q @ I.T)[0])[:5]              # I: catalogue images

shortlist = "\n".join(f"- {cat[j]['sku']}: {cat[j]['captions'][0]}"
                      for j in top5)

prompt = (f'Here are the catalogue entries our search returned for '
          f'"something to sit on":\n{shortlist}\n\n'
          f'Recommend the best ones, citing only SKUs from the list above. '
          f'If none fit, say so.')

answer = generate(prompt)                          # a 0.5B instruction model
cited  = re.findall(r"CAT-\d{6}", answer)
print(f"{sum(s in valid_skus for s in cited)}/{len(cited)} cited SKUs exist")
```

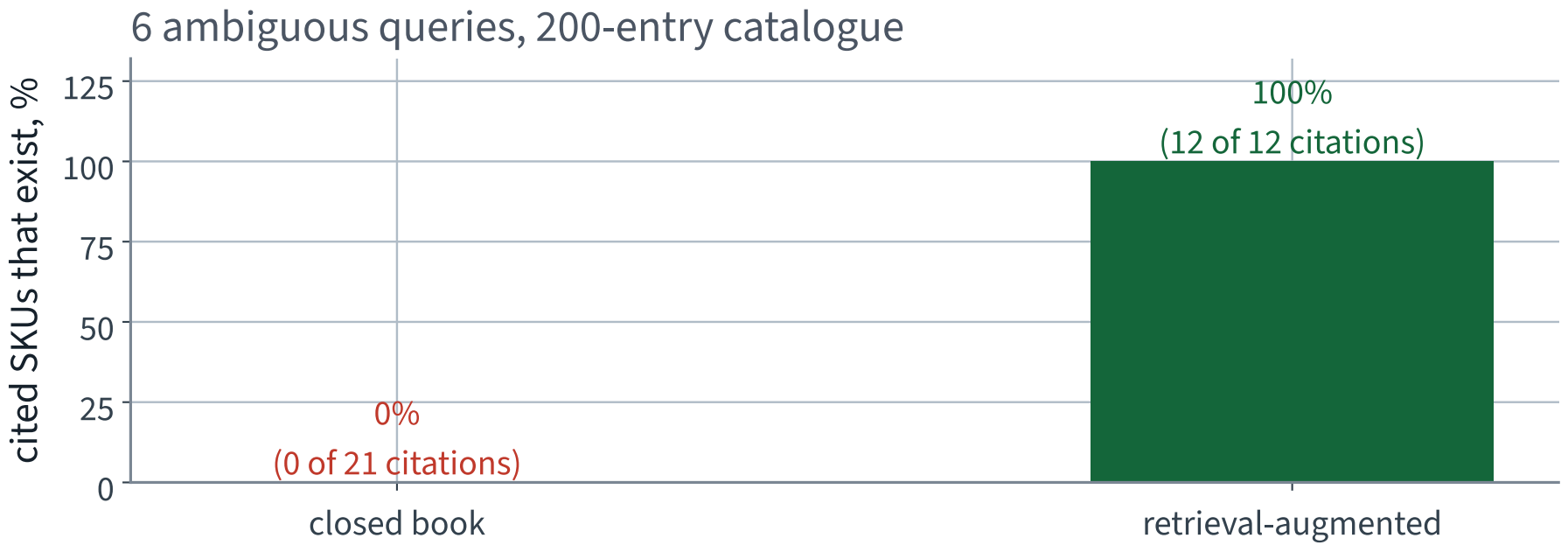
Closed book — what it does without retrieval

ASKED THE SAME QUESTION WITH NO CATALOGUE

The model produces a fluent recommendation and cites stock numbers in exactly the right format. Across 6 queries it emitted 21 citations, of which **0** correspond to an entry that exists.

The failure is not that it refuses. It is that it does not refuse, and nothing in the output distinguishes an invented SKU from a real one.

With the catalogue in the prompt



0% of closed-book citations exist against 100% when the shortlist is in the prompt — over 6 queries and a 200-entry catalogue.

What this does ✗ not fix

- **Grounding is not correctness.** A cited entry can exist and still be a bad recommendation. We measured the cheap half
- **The retriever is now the ceiling.** If the right entry is not in the top five, no amount of generation will recover it — which is why the R@5 we measured last lecture is the number that matters here
- **Fluency is unchanged.** A wrong answer built from real SKUs reads better than a wrong answer built from invented ones

> **WHERE THIS SITS IN THE FIELD**
>
> Evaluating generated answers — faithfulness, attribution, judging with another model — is a live research area and entirely outside Chapters 1–16. Not examinable, and worth knowing exists.

Everything, re-measured

Route	Before	After
Text query → image, joint space	75.0%	75.0% (untouched)
Text index, on the entries with no description	✗ 0%	✓ 60.00%
Ambiguous queries, grounding rate	✗ 0%	✓ 100%

Two repairs, two measured deltas, one number deliberately unchanged as a control. That last row is what makes the other two believable.

The argument

Every method in this course arrived because something measurable could not be done without it.

> **AND EVERY LECTURE HAD THE SAME SHAPE**
>
> A quantity that has to be derived before the method makes sense; the method; a worked example with real numbers; and the conditions under which it fails. Seventeen derivations, and they are the part that will still be true when every library in these notebooks has been replaced.

Partway through, the lectures were cut to what the room actually covered: fewer slides each, and the same four parts in every one.

Thirteen datasets, fourteen first failures

Each application, and the thing that broke first

the red line is the measurement that failed; the method arrived afterwards

1–3 · housing prices
scaled before splitting, scored on the training set
→ pipelines, cross-validation

3 · rare-event detector
90.96% accurate, and it never fires at all
→ precision, recall, thresholds

4–5 · survival probabilities
degree 5 fit the training set and nothing else
→ bias–variance, ridge, lasso

6–7 · land cover
one tree, refit, gives a different explanation
→ impurity, bagging, forests

8 · face grouping
4,096 dimensions, where light moves a face further
→ SVD, PCA, projection

9–10 · garment sorting
a loop you cannot open, and two silent bugs
→ PyTorch, autograd, eval()

11 · deep recognition
twenty layers, and the signal died on the way
→ initialisation, normalisation

12–13 · visual inspection
from scratch, on far too few labelled images
→ convolution, transfer learning

14 · object detection
a visibly wrong box, and no number for how wrong
→ IoU, GIoU, average precision

15–16 · demand forecasting
a random split across time: you forecast the past
→ stationarity, backtesting

17–18 · customer feedback
a word vocabulary the next review escapes
→ subwords, attention

19–20 · claim checking
98% of claims match no document at all
→ BM25, NDCG, dense retrieval

21–22 · recommendation
RMSE on the films they had already chosen
→ factorisation, honest protocols

23–24 · the catalogue
two encoders, two spaces, and nothing between them
→ the contrastive objective, RAG

Read the middle line of each box. That is the syllabus — the chapter order is a consequence of it.

The seventeen derivations, in one place

- Least squares and the normal equation · 2, 3
- Imbalance; precision is not monotone · 3
- Gradient descent · 4
- Variance of a correlated average · 5
- PCA via the SVD · 6
- What a layer computes · 7
- Reverse-mode automatic differentiation · 8
- The Shapley value and its efficiency property · 10
- A linear autoencoder recovers the principal subspace · 11
- Weight sharing and equivariance · 12
- IoU's vanishing gradient; mAP · 14
- Stationarity, differencing, autocorrelation · 15
- Softmax, cross-entropy and logits · 17
- Scaled dot-product attention · 18
- Evaluating a ranking: MRR, AP, NDCG · 19
- Matrix factorisation and the SVD · 21
- The contrastive objective and its temperature · 23

✗ Forty per cent of the written paper, and the part of this course with the longest shelf life.

The failures, collected

Failure	First met	Measured cost
Fitting before splitting	Lecture 3	nothing measurable — and unknowable in advance
Evaluating on training data	Lecture 3	an RMSE of zero
Accuracy under imbalance	Lecture 3	a useless model above 90%
Choosing on the test set	Lectures 3, 5	an optimistic report, every time
Missing `zero_grad()`	Lecture 8	66 points
Ten convolutions, no batch norm	Lecture 12	a network that does not train at all
Random split on a series	Lecture 15	optimistic, and undeployable
Augmenting the validation set	Lecture 13	2.16 points of noise on the criterion

✗ Not one of them raises an exception. That is the whole reason the course is organised around measurement.

The catalogue of silent failures, closed

It runs, and produces a plausible number	Diagnosed in
Scaler fitted before the split	L3
Evaluating on training data	L3
Accuracy under class imbalance	L3
Hyperparameters chosen on the test set	L3, and again on a prompt template in L23
Missing `model.eval()` / `optimizer.zero_grad()`	L8
Metric averaged per batch, not over the set	L8
Random split on an autocorrelated series	L15
Embeddings compared without normalising	L23

✗ Every row of that table runs to completion. Not one of them raises.

Gaps: what this course did not cover

- **Reinforcement learning** — an agent acting, and learning from the consequences. A different problem shape entirely
- **Generative image models** — diffusion, and what it optimises
- **Causality** — every method here predicts; none of them tells you what happens if you *intervene*
- **Fairness and model governance** — who a model is worse for, and who is accountable when it is
- **Serving and drift** — the model in production, and how it decays quietly once the world moves

Named so you know they exist and roughly where they sit. None is examinable, and each is a course.

The two you are most likely to need first

Drift

Every number in this course was measured on data drawn like the training data. In production that stops being true, gradually and without warning, and the score you reported at launch is the last honest one anybody has.

Monitor the inputs, not just the outputs: the distribution moves before the accuracy does.

Who it is worse for

Lecture 22 asked whom a recommender serves: popularity alone scores respectably, and the metric gives a film everyone has heard of the same credit as a discovery. Asking that costs a line of code, and it is the difference between a system you can defend and one you cannot.

The habit generalises: always ask for whom the average is wrong.

Gaps that are beyond the book entirely

Topic	Why it matters, in one line
Causal inference	every model here answers “what is associated with what”; deciding an *intervention* needs different assumptions and different data
Uncertainty quantification	Lecture 4 checked whether one classifier’s probabilities were calibrated. Predictive uncertainty in general is its own subject
Fairness and harm	we asked once, in Lecture 22, whom a system serves. Who it fails, and what that costs them, is not a metric you can pick off a list
Privacy	differential privacy, federated learning, and what a model memorises about its training data
Monitoring and drift	a model is fitted once and lives for years. Detecting that the world moved is a permanent job
Experimental design	an offline metric improving is not a business improving. That is an A/B test, and it is a statistics problem

What you can do now

- Take a problem, frame it, and say what a correct answer would be *before* fitting anything
- Choose a metric that matches the brief, and measure the trivial baseline first
- Build the simplest thing that runs, inside a pipeline, evaluated on a protocol you fixed in advance
- Read the failure: slice the error, look at the worst cases, and name the conditions under which the method breaks
- Improve one thing at a time, and report the ablation rather than the best row
- Write down what the number was measured on, every time

Nothing in that is specific to a library, and none of it will be out of date in ten years.

A last word on the assistant

Much of the code you write from here on will be drafted with an assistant's help. That changes who types it. It does not change what this course was about.

> **WHAT AN ASSISTANT CANNOT SUPPLY**
>
> Whether the number at the end means anything. "What was this measured on?", "what is the baseline?", "was anything fitted before the split?" — those questions come from knowing the material, and every lecture of this course was built around them.

✗ The code was never the scarce part. Knowing whether to believe the output is, and that is what these twenty-four lectures were for.

The examination

Written — two hours, closed book

- Technical exercises, closed choices, short open answers
- A · the derivations (40%)
- B · choosing a method for a stated situation (35%)
- C · reading results (25%)

Oral — optional, seven to ten minutes

- Three questions, on any topic from the course
- No notes, no computer
- You decide after seeing your written mark

The written is marked out of 30, pass at 18, capped at 27 on its own; the oral moves it up by at most three marks or down by any amount, below 18 included, and 28, 29 and 30 exist only through it. Everything in Chapters 1–10 and 12–16 and in the lecture notes is examinable; nothing marked *engineering* or *for context* is. The 120 exercises that end these decks are published with their solutions.

How to keep learning

- **Reproduce something.** Take a paper's headline number and try to get it. You will learn more from the gap than from the paper
- **Keep the habit of the baseline.** It is the single cheapest defence against a wrong conclusion, and almost nobody does it
- **Read the evaluation section first.** If it does not say what was held out and how, the rest is decoration
- **Write the check before the model.** Every notebook in this course does, and it is why they still run

The libraries in these notebooks will be superseded. The derivations will not, and neither will the habit of asking what a number was measured on.

The notebook for this lecture

Open Lecture 24 in Colab

- **Runs on free CPU.** Small pretrained models, inference only
- Captions generated for the entries that have none, and the retrieval re-measured with them
- The closed-book control: the same questions with retrieval off, so what retrieval bought is a number

> **WHAT TO CHANGE**
>
> Set `do_sample=True` and run the grounding measurement three times. The percentage moves every run: a sampled generation makes it a random variable whose spread nothing here has characterised.

Design exercise · in the manner of an ML system-design interview

The task: a developer-tools company's documentation assistant

> **THE BRIEF**
>
> The company sells a payments API. On its documentation site a developer asks "how do I rotate a key without downtime?" and the assistant must answer **in words, from the company's own documentation, citing the pages it used** — or say that the documentation does not answer. The pages change at every release; some are visible only to certain customers. The business measures tickets deflected, time to a first successful call, and the cost of a developer shipping what a wrong answer said.

- **Given:** the documentation — guides, reference pages, changelogs, each with an identifier and a visibility — and the support channel's questions, a sample judged by the writers
- **Scale:** tens of thousands of pages, thousands of questions a day
- **Consumers of the output:** the developer asking, the support team, the writers

✓ Nine steps follow, each with the pitfall that catches a first attempt and what to do about it. The model at the

Step 1 — Frame it: decide how you will check it before anything is built

- “The answer is good” is not measurable. What is: every citation names a page that **exists**; the cited passage **supports** the sentence it is attached to; and when nothing relevant is retrieved, the assistant **declines**
- Two baselines, both bad on purpose: show the top ten passages and let the developer read, which is what the docs search already does; and ask a language model with no documentation, which is fluent and invented
- End-to-end correctness is judged last, by the writers, on a sample

> **PITFALL**
>
> Measuring grounding and reporting it as correctness. A cited page can exist, support the sentence, and still be the wrong procedure for the developer’s API version, and the launch post quotes the number. Grounding is the cheap half and the half a machine can check; say which half a number measures, every time.

Step 2 — The corpus: chunk it, and treat the chunking as a hyperparameter

- Before anything is retrieved the documentation is cut into units, each carrying its page's identifier, title, owner and visibility — a choice upstream of both halves, usually made without measurement
- Too small and a chunk lacks the context that makes it answerable; too large and its embedding averages several topics and matches none; split badly and a code sample is cut from the sentence that explains it
- Judged questions: a sample from the support channel, each marked by the page's writer with the page that answers it — or with the fact that none does

> **PITFALL**
>
> Picking a chunk size by feel and never revisiting it. Vary it with everything else held fixed and measure recall at k on the judged questions — the retrieval protocol, unchanged — then cut at the page's own headings, so that a procedure's steps and its code sample stay together and a chunk can be shown as a readable passage.

Step 3 — Split the judged questions, and choose the prompt on the validation half

- Validation questions for every choice — chunk size, how many to retrieve, the re-ranking depth, the wording of the prompt; test questions read once, before the release
- Include questions whose answer is genuinely absent from the documentation, in both halves, so that abstention is measured deliberately rather than hoped for
- Fix the decoding: a sampled generation turns every rate into a random variable, so decode deterministically or report the spread over repeated runs

> **PITFALL — THE TEMPLATE CHOSEN ON THE TEST SET**
>
> Trying several prompt wordings and reporting the one with the best grounding rate on the test questions is a hyperparameter chosen on the test set, with a sentence in place of a number — the same error met with a numeric hyperparameter and again with a zero-shot prompt. Choose the template on the validation questions; report it on the test, once.

Step 4 — The retriever, and the ceiling it sets

- A bi-encoder over the chunks, fused with a lexical index for the endpoint names and error codes a vector space does not guarantee to match; a cross-encoder re-ranks the top
- Retrieve a generous k with the cheap first stage and pass only the top few on: high recall from the first stage, high precision from the second, which is where the two-stage argument earns most
- Measure the retriever on its own, by recall at the k the generator will see, on the judged questions

PITFALL

Tuning the generator while the right chunk is not in the context. If the page that answers is not among the k retrieved, no amount of generation recovers it: the retriever's recall at k is the ceiling of the whole system, and a quarter spent on prompts under a low ceiling is lost. Read that number first, and spend on the retriever — chunking, hybrid, re-ranking — before touching the prompt.

Step 5 — The generator, and the failures that belong to the join

- The retrieved chunks go into the generator's context with the question and an instruction: answer from these pages only, cite their identifiers, and say so if none of them answers
- The knowledge lives in the index, not in the weights: a page added at a release is answerable at once; a deprecated one stops being cited; nothing is retrained
- Each half covers the other's weakness — the retriever only ranks, the generator will answer regardless

PITFALL — THREE FAILURES VISIBLE FROM NEITHER HALF

Retrieved but ignored: the right page is in the context and the answer comes from the weights, an older API. **Not retrieved, answered anyway:** the retriever misses and the generator does not say so. **Retrieved and misread:** the page is present and correct and the answer misstates it. Recall at k and fluency can both look excellent while the system does all three; the join needs its own measurement.

Step 6 — Evaluate the join: the mechanical check, and the closed-book control

- Extract every identifier the answer cites and count those that exist: the grounding rate, with no judgement call in the loop. Then, on a sample, whether each cited passage supports its sentence: attribution, checked by a writer
- Run the same questions with retrieval off. Whatever the assistant still gets right it knew already, from the old docs on the public web; the difference is what retrieval bought
- On the absent-answer questions, count the abstentions

> **PITFALL — A CITATION THAT LOOKS RIGHT**
>
> Closed book, the model produces a fluent answer and cites pages in exactly the right format, none of which exists; nothing in the output tells an invented citation from a real one, and a developer without the docs open cannot tell either. The check has to be mechanical — identifier against the index, then passage against the claim — and a citation that does not support its sentence is worse than none.

Step 7 — Access control: the index holds everything, the context must not

- Every chunk carries its page's visibility: public, private beta, or internal. The retriever filters by the asker's entitlements *before* ranking, so that a chunk the developer may not read never reaches the generator
- The generator cannot keep a secret it has been shown: whatever is in the context can appear, quoted or paraphrased, in the answer
- Test it deliberately: questions whose answer lies only in a page the asker cannot see, where the required behaviour is to decline

PITFALL — LEAKAGE INTO THE ANSWER

One index over everything, filtered after generation or not at all. An unannounced feature from a private-beta page is retrieved because it matches the question well, and the assistant describes it, correctly grounded, to a customer who should not know it exists. Filter at retrieval, log which pages each answer was built from, and keep the cannot-see questions in the test set, so that a leak is a failed test rather than an incident.

Step 8 — In production: the docs move at every release, so the number moves

- Re-index on every release, and re-run the judged questions after each rebuild: recall at k, grounding, abstention. A drop is the regression, and where a renamed page shows up
- Roll out to a share of developers first; monitor the live answers — grounding rate by day, citations to pages that no longer exist, abstentions, the tickets that follow an answer — with an engineer on call for the index
- Route the questions with no supported answer to the writers: they are the list of what the documentation does not say

> **PITFALL**
>
> Treating the launch number as permanent. The score reported at launch was measured on that release's documentation; pages are rewritten, moved and withdrawn, the questions drift with the product, and the assistant decays quietly because nothing raises an exception when a cited page stops existing. Monitor the inputs and the citations, and refresh the judged set as the documents are.

Step 9 — Ship it: the pipeline, and the pitfalls in one table

Documentation → chunks at the headings, each with its identifier and visibility → judged questions split, absent answers included → hybrid retriever, re-ranked, recall at k first → generator answering from the context only, with citations → grounding checked mechanically, closed-book control → entitlements filtered at retrieval → re-index at every release, monitor.

Pitfall	Address it by
Grounding reported as correctness	naming which half a number measures
A chunk size chosen by feel	recall at k on the judged questions, chunking varied alone
The prompt chosen on the test questions	the template chosen on validation; decoding fixed
The generator tuned under a low ceiling	the retriever's recall at k read first
Failures of the join unseen	a measurement of the join, not of the halves
Invented citations in the right format	the mechanical check; the closed-book control
A private-beta page in the answer	entitlements filtered at retrieval

What we did today

1. Wrote captions for the 60 entries that had none, and recovered R@1 on them from 0% to 60.00%, against 66.67% had a person written them
2. Joined a retriever to a generator, and located the failures in the *join* rather than in either half
3. Made the grounding check mechanical, and moved the assistant's citation validity from 0% to 100% — grounding, not correctness
4. Read the course as one argument: seventeen derivations, and a catalogue of failures none of which raises an exception
5. Named what the course did not cover, and how to keep learning past it
6. Designed a developer-tools company's documentation assistant in nine steps — the check decided first and grounding kept apart from correctness, chunking measured as a hyperparameter, the prompt chosen on validation, the retriever's recall read as the ceiling, the three failures of the join, a mechanical citation check with a closed-book control, entitlements filtered at retrieval, and the judged set re-run at every release — each step named for the pitfall it avoids

FAILURE CONDITION — A CITATION THAT LOOKS RIGHT

A generated answer can cite a SKU that does not exist, or one that exists and does not support the sentence. Both read as citations. The check has to be mechanical — match the SKU against the catalogue, then the passage against the claim — because a reader without

If you remember one thing

> **THE LINE**
>
> **A number is only as good as the question “measured on what?” can be answered.**

Every failure in this course — the leak, the training score, accuracy under imbalance, the random split on a series, the augmented validation set, the ungrounded generated answer — is a case where that question had an answer nobody had asked for.

Asking it costs seconds. Not asking it has cost, in these twenty-four lectures alone, an RMSE of zero, a 90% classifier that never fires, a network that does not train, and a forecast of the past.

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 24

five, in the style of the written paper

Exercises — Lecture 24

1 Sixty catalogue entries have no description. State why their text-route R@1 of 0% is structural rather than statistical, and what fraction of the loss an auto-caption recovered. [5]

2 Name the three failures that belong to the join of a retriever and a generator, and state why evaluating each half alone does not see them. [5]

3 A retrieval-augmented system is measured by whether cited stock numbers exist. State what that measures, and what it does not. [4]

Solutions overleaf — there is no Lecture 25.

Exercises — Lecture 24 (continued)

4 In a retrieval-augmented pipeline the retriever's recall bounds the whole system. Explain why, and what that implies about where to invest. [4]

5 Name three rows of the course's catalogue of silent failures, and state the property they share and what it implies about how such failures are found. [5]

Solutions overleaf — there is no Lecture 25.

THE FIVE SET AT THE END OF LECTURE 23

Solutions Lecture 23

not part of this lecture

Solutions — Lecture 23

1. Two encoders are trained separately, one on images and one on text. Explain why their cosine similarity…

✓ Any rotation of the image space leaves every image–image cosine and the image loss unchanged, while changing every image–text cosine.

- The objective cannot distinguish the rotated space from the original
- So it did not pick one, and the relative geometry is arbitrary

2. A rank is computed by counting only the candidates that score strictly higher than the right answer. State…

✓ 100%: every candidate ties with the right answer and a strict inequality places every tie behind it. Count with $\geq$, so ties count against the model.

- A model that has learned nothing, or a bug that zeroed the embeddings, then looks perfect
- With ties counted against it the same matrix scores 0%

Solutions — Lecture 23 (2)

3. The InfoNCE logits are cosines divided by a temperature τ. State what τ cannot change for a fixed…

✓ It cannot change which column of a row is largest, so not the ranking. It changes where the gradient goes: $(p_{ij} - \mathbf{1}[j = i])/\tau$.

- Dividing by a positive τ is monotone, so every row keeps its order
- A small τ concentrates the push on the few hardest negatives

4. Recall@10 is reported over 200 candidates. State the exact random baseline, and why it is computed rather…

✓ $10/200 = 5\%$.

- One relevant item ranked uniformly gives $P(\mathrm{rank} \le k) = k/n$
- A closed form needs no seed and carries no noise

Solutions — Lecture 23 (3)

5. An entry with no written description is never returned by a text index built over the descriptions. State…

✓ Structural — an entry with no text is not in a text index at all, so the repair is to match the query against the picture instead.

- No amount of re-ranking reaches a document that was never indexed
- The jointly trained dual encoder scores the query against the image, and never sees a description

ANSWERED HERE, BECAUSE THERE IS NO LECTURE 25

Solutions Lecture 24

not part of this lecture

Solutions — Lecture 24

1. Sixty catalogue entries have no description. State why their text-route R@1 of 0% is structural rather than…

✓ A text index cannot rank a document that does not exist, so those entries score zero whatever the queries. Captions bring R@1 on them to 60.00%, against 66.67% with a person's description: part of the loss, not all of it.

- No choice of queries or seed moves a structural zero; only adding text does
- Report the part recovered, measured on the same 60 entries, not a direction

2. Name the three failures that belong to the join of a retriever and a generator, and state why evaluating each…

✓ Retrieved but ignored; not retrieved, answered anyway; retrieved and misread. Recall@k on the retriever and fluency on the generator can both look excellent while the system does all three.

- Each failure needs both halves present to occur
- So the join needs its own measurement: groundedness, attribution, abstention, then end to end

Solutions — Lecture 24 (2)

3. A retrieval-augmented system is measured by whether cited stock numbers exist. State what that measures, and…

✓ It measures grounding, not helpfulness.

- A cited entry can exist and still be a bad recommendation
- It is the cheap half, and saying which half was measured is the point

4. In a retrieval-augmented pipeline the retriever's recall bounds the whole system. Explain why, and what…

✓ If the right entry is not retrieved, no generation recovers it.

- The generator can only work with what it is given
- The first stage sets the ceiling, exactly as in Lecture 20

Solutions — Lecture 24 (3)

5. Name three rows of the course's catalogue of silent failures, and state the property they share and…

✓ For example a scaler fitted before the split, a missing `zero_grad()`, a random split on an autocorrelated series. None of them raises: each runs to completion and produces a plausible number.

- So no error message will ever point at them
- They are found only by measuring — a baseline, a protocol fixed in advance, a check written before the model

APPLICAZIONI INFORMATICHE DEL MACHINE LEARNING

Thank you

Seventeen derivations · twenty-three notebooks · thirteen datasets
Every number on every slide reproduced by a script in the repository

Questions, and then the examination